Cross-Domain Entity Matching with Robust Optimization

ABSTRACT

Entity Matching (EM), the task of identifying records across data sources that represent the same real-world entity, is fundamental to data management. Recent EM systems built by fine-tuning Pretrained Language Models (PLMs) and Large Language Models (LLMs) achieve strong overall accuracy, yet their performance can vary considerably across domains, with smaller or underrepresented domains often performing substantially worse. We address this limitation by formulating robust domain-aware EM, which seeks to strengthen underperforming domains without sacrificing overall matching quality. We propose RoDo-EM, a robust optimization framework that automatically adjusts the intensity of robustness-aware training in response to observed performance across domains. This adaptive weighting mechanism is integrated with robustness-aware training objectives, domain-aware sampling, and regularization to learn a single EM model that performs reliably across heterogeneous domains. Through extensive evaluation on multiple domain-partitioned EM benchmarks, we show that RoDo-EM improves robustness across domains while preserving competitive aggregate performance and outperforms state-of-the-art EM approaches across a range of robustness objectives.

ACM Reference Format:

. 2018. Cross-Domain Entity Matching with Robust Optimization. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Entity Matching (EM) is a longstanding problem in data integration and a fundamental operation in data processing pipelines. Also known as Entity Resolution or Record Linkage, EM is the task of determining whether a given pair of records refers to the same real-world object [12, 13]. These records may consist of structured rows in a table with a fixed schema or more loosely structured forms of data. Furthermore, the records may originate from the same table–for example, to perform deduplication–or from different sources, as in the traditional data integration setting [66].

EM has a rich history in the database community, with continual advances in methods, problem settings, and applications. Early EM research focused on traditional string-similarity and probabilistic approaches [18, 65], primarily for record linkage over relatively small databases. Over time, rule-based methods emerged [23, 34]

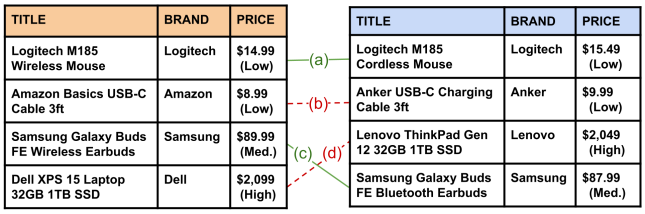


Figure 1: EM example with various price categories.

followed by learning-based approaches [7, 8], and deep neural network models [17, 35]. More recently, the dominant paradigm has shifted towards fine-tuning pretrained models on task-specific data, particularly Pretrained Language Models (PLMs) [10, 28], such as BERT [14] and RoBERTa [31], as well as Large Language Models (LLMs) [43], such as Llama [15]. Although LLMs have demonstrated strong zero-shot performance when labeled training data is scarce, fine-tuned PLMs remain the prevailing approach for EM [3], largely due to reduced inference latency and increased matching accuracy for targeted EM tasks.

In recent years, the increasing scale of pretrained models has enabled them to (1) achieve higher matching accuracy and (2) learn increasingly general representations of the EM task [11, 61]. This progress has led to the emergence of *domain-aware EM* [45, 53, 55], in which EM is viewed as a collection of related but distinct domains or subtasks. Existing work has explored domain adaptation, where a model learns to perform matching on a target domain by leveraging information from one or more source domains [53, 55]. More recently, researchers have also investigated task-specific EM [45], where multiple models are specialized for individual subtasks within a broader application domain (e.g., "Computers" and "Electronics" within the larger "E-commerce" domain). While these methods achieve state-of-the-art performance on domain-aware EM tasks, they rarely consider training a *single general-purpose EM* model that performs well across multiple domains [59]. In practice, such a model offers several advantages, including reduced maintenance, lower training cost, and decreased energy consumption compared to maintaining multiple specialized models.

LLMs in zero-shot settings can serve as general EM models across domains [11, 43]. However, these approaches present two important limitations. First, they incur substantially higher *inference latency, energy consumption, and deployment cost (e.g., when using hosted APIs)*, partially offsetting the practical advantages of a single general model. Second, and more importantly, they do not explicitly optimize for worst-case performance across domains [59, 60]. We use the term *robust* to refer to models that explicitly optimize the performance of their worst-performing domain, rather than only maximizing average matching performance. Note that this notion of robustness is inspired by a line of prior work in information retrieval that characterizes robustness in terms of a system's effectiveness on the most difficult topics [38, 58, 62]. Existing EM techniques, including the use of LLMs, may achieve strong overall performance

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

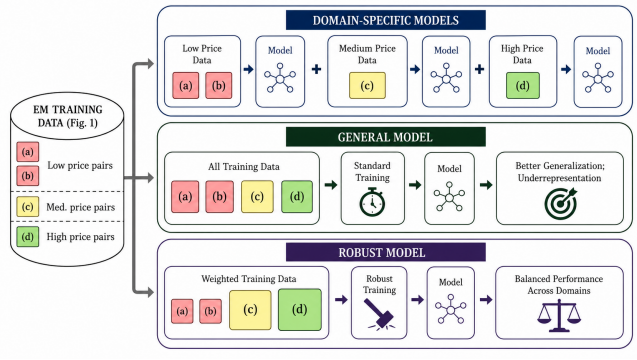


Figure 2: The specific, general, and robust EM models for the toy dataset of Figure 1.

while underperforming on smaller or underrepresented domains, providing no guarantees on worst-case domain performance.

In this paper, we study the problem of training a robust general-purpose domain-aware EM model that combines language model finetuning with optimization objectives that account not only for overall matching performance, but also for performance on the most challenging domains. To motivate this problem setting, we first consider an illustrative example.

Example 1.1. *Consider a data engineer at a large e-commerce company who must match product listings against an existing product catalog. The workload consists of products from three application-defined price ranges: "Low" (<$50), "Medium" ($50–$300), and "High" (>$300), where low-priced products constitute the majority of the data and high-priced products represent a relatively small minority. For simplicity, we assume the listings and catalog share the same schema.*

The engineer first applies blocking *to eliminate unlikely matches, producing a set of* candidate pairs *for the matching stage. Figure 1 shows several representative candidate pairs: (a) and (b) belong to the "Low Price" domain, (c) to the "Medium Price" domain, and (d) to the "High Price" domain. A solid green line denotes a ground-truth match, while a dotted red line denotes a non-match.*

Having introduced domain-aware EM in the context of prices, we now motivate the need for *robust* optimization by discussing underperforming domains importance in practice.

Example 1.2. *The engineer's objective is not simply to train a model that achieves high matching accuracy* on average. *Such a model may perform exceptionally well on the abundant low-priced products while performing considerably worse on the relatively scarce high-priced products. In practice, however, the company is particularly concerned with ensuring that performance remains* robust *across all price ranges. Specifically, it seeks to maintain strong performance on the "High Price" domain, where incorrect predictions may have substantially greater financial consequences than errors on lower-priced products. For example, if pair (d) is incorrectly classified as a non-match, the company may lose the sale of a $2,000 product. In contrast, making the same mistake on pair (a) may only affect a $15 item. This example illustrates why maintaining strong performance on the "High Price" domain is important despite its relatively small representation in the training data.*

There are several possible training strategies for the EM scenario introduced in Example 1.2, as illustrated in Figure 2. A straightforward approach is to train a separate model for each domain, fine-tuning one model for "Low Price," one for "Medium Price," and one for "High Price" (top of the figure). While intuitive, this approach requires training and maintaining multiple models, and domains with relatively few training samples, such as "High Price," may still exhibit poor matching performance. The middle portion of the figure illustrates an alternative strategy in which all labeled samples are used to train a single general-purpose model. Although this approach is simpler and allows information to be shared across domains, the resulting model may underperform on minority or inherently more challenging domains because optimization primarily favors overall performance. Finally, the bottom portion of the figure illustrates the proposed *robust* general model. By incorporating robust optimization techniques such as sample weighting and regularization, the model leverages all available training data while explicitly improving performance across domains despite differences in domain size and matching difficulty.

Consequently, the engineer in Example 1.2 would prefer to train this robustness-aware general model but must determine how best to optimize it to achieve strong performance across all domains. This is precisely the objective of our work: to develop a robust domain-aware EM framework, while also investigating alternative domain-level optimization objectives. In pursuit of this goal, our paper makes the following contributions:

- *Problem Formulation (Section 3).* We formalize the problem of *Robust Entity Matching Across Domains* (REMAD), extending traditional EM to account for general-purpose models that optimize domain-level robustness across multiple domains.
- *Robust Optimization for EM (Section 4).* We investigate several robust optimization techniques for domain-aware EM, including Distributionally Robust Optimization (DRO) [6], and study their effectiveness for improving performance.
- *RoDo-EM (Section 4.4).* We introduce **RoDo-EM**,[1] a unified framework for robust domain-aware EM that integrates a novel adaptive weighting mechanism with robustness-aware optimization objectives, domain-aware sampling, and regularization. Our adaptive mechanism dynamically adjusts robustness strength based on the distribution of domain-level training losses, eliminating the need to manually tune objective-specific robustness weights.
- *Experimental Evaluation (Sections 5-6).* We evaluate robust optimization by fine-tuning PLMs and LLMs on general EM tasks and comparing against state-of-the-art baselines. To support evaluation, we derive 4 general EM benchmark tasks with diverse domain distributions, which we also release to facilitate future research on robust domain-aware EM.

2 RELATED WORK

Our work builds upon recent advances in machine learning and deep learning for Entity Matching (EM) [4, 17, 35], particularly approaches based on Pretrained Language Models (PLMs) [3, 28] and Large Language Models (LLMs) [43, 52]. While these learned techniques have also been applied to other stages of the Entity

[1]Tech. Report: https://anonymous.4open.science/r/RODO-EM-8350/tech_report.pdf.

Resolution pipeline, such as blocking [9, 25], this work focuses exclusively on the matching stage.

Among PLM-based approaches, DITTO [28] was one of the first methods to demonstrate the effectiveness of fine-tuning pretrained language models for EM. Since its introduction, the DITTO pipeline has served as the foundation for subsequent systems [10, 20, 21, 41, 69], including AdapterEM [36], MultiMatch [37] and BEACON [45]. Accordingly, we adopt the DITTO implementation as the foundation for our experimental framework.

More recently, Large Language Models (LLMs) have emerged as a promising approach for EM. Early work explored both open-source and hosted LLMs for zero-shot and few-shot matching [43], while subsequent studies proposed fine-tuned LLMs for improved task-specific performance [52, 68]. Prompting-based methods have likewise become increasingly popular, leveraging in-context learning, retrieval augmentation, and specialized prompting strategies to improve matching accuracy [11, 19, 22, 63]. Although these approaches have demonstrated improved matching accuracy in several settings, their higher inference latency and computational cost have thus far limited widespread adoption in practice [3].

Domain-Aware Entity Matching. Recent work has explored domain-aware learning for EM, where a domain corresponds to a particular type of data on which the model performs matching [57, 67]. The predominant line of research in this area is *domain adaptation*, where a model learns to perform EM on a target domain with limited or no labeled data by leveraging a source domain with abundant labeled examples. Early work focused on deep learning approaches, including Thirumuruganathan et. al. [54], who proposed a distributed representation for record pairs, and Kasai et. al. [26], who combined transfer learning with active learning to address low-resource EM settings. DAME [55] introduced a mixture-of-experts framework to capture domain-specific knowledge from multiple source domains and transfer it to a target domain. DADER [56] further formalized domain adaptation for EM as an end-to-end framework by introducing a feature alignment module designed to reduce distributional shift between source and target domains. More recently, MFSN [53] distinguishes between shared and domain-specific matching features, while Okayama et. al. [40] investigate domain adaptation for prompt-based and LLM-based EM.

Although these methods demonstrate the importance of domain structure in EM, domain adaptation differs fundamentally from the *EM across domains* problem considered here. Domain adaptation transfers knowledge from source domains (e.g., "Computers") to a target domain with limited labeled data (e.g., "Cell Phones"), whereas EM across domains leverages all available data to improve matching across domains. BEACON [45] formally introduced this setting and proposed distribution-aware sampling strategies for training domain-specific EM models under a fixed labeling budget. Building on this foundation, we address a different optimization problem: training a single general-purpose EM model that achieves robust performance across domains, rather than selecting training data for multiple domain-specific models. Specifically, we investigate objectives that improve lower-bound domain performance and domain parity while preserving strong overall performance.

Robust Optimization Techniques. Robust optimization for EM has received relatively limited attention [1, 28, 33]. Existing approaches primarily define robustness as strong performance in the context of label imbalance and distribution shift in EM datasets. To the best of our knowledge, however, no prior work has explicitly investigated domain or group-level robustness for EM, which is the focus of this paper. In contrast, robust optimization across groups has become an active area of research in the broader machine learning community [16, 29, 39, 48].

Sagawa *et al.* [48] introduced Group Distributionally Robust Optimization (Group DRO), extending classical Distributionally Robust Optimization (DRO) [6] to optimize worst-group performance during training. They further demonstrated that regularization and early stopping are critical for achieving strong robustness across groups. Liu et. al. [29] extended this framework to settings where group labels are unavailable by learning to identify minority groups during training. More recently, Nguyen et. al. [39] studied Group DRO in the context of domain generalization, introducing Risk Distribution Matching as a mechanism for encouraging balanced performance across domains.

Our work builds upon this line of research by applying group-based robust optimization techniques to the EM across domains setting. Specifically, we investigate how robust optimization can be used to train a single general-purpose EM model that improves worst-domain performance while maintaining strong overall matching accuracy.

Algorithmic Fairness. Closely related to robust group optimization is algorithmic fairness, which seeks to ensure equitable performance across demographic groups, often through group-level parity objectives [5, 32, 47, 64]. Fairness has also received growing attention in the EM community [2, 30, 51], primarily focusing on matching tasks involving people and disparities across sensitive demographic groups.

Our work is related to fairness in both machine learning and EM but differs in two important respects. First, fairness research typically focuses on groups defined by sensitive attributes. In contrast, this work considers non-sensitive application domains, such as product categories or price ranges, where the objective is to improve performance across operational domains rather than demographic groups. Second, although RoDo-EM includes domain-parity objectives inspired by the fairness literature, parity is only one notion of robustness within our framework. Depending on the application, practitioners may want to optimize for worst-case domain performance or domain parity while maintaining strong overall matching accuracy.

3 PRELIMINARIES

We now introduce the notation and formal definitions used throughout the remainder of the paper, formulate the standard EM task (Section 3.1), extend it to the domain-aware EM setting (Section 3.2), and formally define the problem of Robust Entity Matching Across Domains (REMAD, Section 3.3).

3.1 Entity Matching

Following Li *et al.* and subsequent work [28, 45], we define EM as the task of matching records between two distinct datasets. Alternative formulations also exist, such as matching records within a single dataset for deduplication [50]. Let D_{left} and D_{right} denote the two datasets to be matched. We assume that each dataset is tabular and

consists of *entities* (or records), where an entity r is represented as a row of structured information. We focus on the *clean-clean* setting, which assumes that neither D_{left} nor D_{right} contain duplicate records internally. The set of all possible record pairs is given by the Cartesian product $D_{\text{left}} \times D_{\text{right}}$. We refer to these pairs as *candidates*, and to the complete collection of pairs as the *candidate set*.

The EM pipeline typically consists of two stages: blocking followed by matching. During the blocking phase, lightweight heuristics are applied to eliminate unlikely matches from the candidate set, thereby reducing the search space. This process produces a subset of candidates $N \subset D_{\text{left}} \times D_{\text{right}}$ such that $|N| \approx \max(O(|D_{\text{left}}|), O(|D_{\text{right}}|))$, avoiding the $O(|D_{\text{left}}| \times |D_{\text{right}}|)$ complexity of exhaustive pairwise comparison. While blocking is essential for scalability (and may also be used to construct domain partitions), our methods focus on improving the downstream matching stage.

We assume that the candidate set N can be partitioned into j disjoint subsets, each corresponding to a distinct domain, where

$$N = n_1 \cup n_2 \cup \cdots \cup n_j, \quad n_i \cap n_k = \emptyset \text{ for } i \neq k.$$

We assume that each candidate pair $(r_1, r_2) \in n_i$ consists of two records that both belong to domain i. Thus, domains partition candidate pairs rather than individual records. Throughout this work, we consider product data, where domains may correspond to product categories (e.g., "Computers," "Clothing"), brands (e.g., "Apple," Samsung"), or price ranges (e.g., "High," "Low").

Given a filtered candidate set N, matching determines which candidate pairs correspond to matching entities. This is formulated as a binary classification problem: for each candidate pair $(r_1, r_2)_i \in N$, we assign a label $y_i \in 0, 1$, where $y_i = 1$ if r_1 and r_2 refer to the same real-world entity, and $y_i = 0$ otherwise. We refer to a candidate pair together with its corresponding label as a *sample*.

To perform this classification, we adopt models such as BERT [14], DistilBERT [49], and Llama [15]. These are transformer-based models pretrained on a large text corpus which we can fine-tune for downstream tasks such as EM. Following DITTO [28] and follow up work [20, 21, 36, 37, 41, 45, 69], an entity r with p attributes is serialized as: `[COL]` $\texttt{attr}_1$ `[VAL]` $\texttt{val}_1 \cdots$ `[COL]` $\texttt{attr}_p$ `[VAL]` $\texttt{val}_p$ and each candidate as: `[CLS]` r_1 `[SEP]` r_2 .

The serialized candidate pair is then provided as input to the model. The `[SEP]` token marks the boundary between the left and right entities, while the `[CLS]` token serves as a learned representation of the entire candidate pair. The resulting `[CLS]` embedding is passed through a fully connected layer, followed by an activation function, to produce the predicted match label $\hat{y}_i$.

3.2 Entity Matching Across Domains

While traditional EM optimizes performance over the entire candidate set, this work explicitly considers the partitioning of candidates into domains and studies optimization objectives defined over these domain-level partitions. Following BEACON [45], we denote the set of domains by $\mathcal{D}$, where $|\mathcal{D}| = j$. Given this formulation, there are two primary approaches for incorporating domain information during training: (1) training separate *domain-specific* EM models for each domain or (2) training a single *general-purpose* EM model using data from all domains.

For the domain-specific approach, we train j independent models, where the training set X_i for domain $i \in \mathcal{D}$ is given by $X_i = n_i$. That is, each model is trained exclusively on the samples belonging to its corresponding domain. As discussed in Section 1, this approach requires one model per domain and may underperform when certain domains contain relatively few labeled examples.

In this paper, we instead focus on training a *single general-purpose* EM model. In this setting, the single training set X consists of the union of all domain partitions:

$$X = N = n_1 \cup n_2 \cup \cdots \cup n_j.$$

Consequently, a single model is trained using samples from all the domains. While this approach improves scalability and enables information sharing across domains, it also introduces a new challenge. Different domains often exhibit distinct schema structures, attribute distributions, data quality, and matching characteristics, causing some domains to be inherently more difficult than others. For example, certain domains may contain noisier attribute values, greater lexical ambiguity, or more subtle distinctions between matching and non-matching records. As a result, a general-purpose model may achieve strong overall performance but consistently underperform on these more challenging domains, even when they are well represented in the training data. Addressing this challenge is the primary motivation for the robust optimization framework.

3.3 Problem Definition

We now extend the formulation of training a general-purpose domain-aware EM model to the *Robust Entity Matching Across Domain* (REMAD) setting. Let θ denote the parameters of the general-purpose EM model, and X the set of labeled pairwise samples available for training. Commonly, a cross-entropy loss is used to perform binary classification over X to optimize θ, and we denote this loss as $\mathcal{L}(\theta)$. The corresponding F1 score, the standard evaluation metric used in EM, is denoted by $F(\theta)$. We further define the corresponding domain-level metrics. For each domain $i \in \mathcal{D}$, we define $\mathcal{L}_i(\theta)$ as the average binary cross-entropy loss over the samples in n_i, and $F_i(\theta)$ as the corresponding domain-level F1 score.

Definition 1 (Robust Entity Matching Across Domains (REMAD))**.** Given a training set X consisting of a domain partition based on $\mathcal{D}$, the objective of REMAD is to train a robust general-purpose EM model θ^* such that

$$\theta^* = \arg\max_{\theta} \min_{i \in \mathcal{D}} F_i(\theta); \quad F(\theta^*) \approx \max_{\theta} F(\theta).$$

Our goal is therefore to *optimize the worst-domain performance of a general-purpose EM model while preserving strong overall matching performance*. This formulation seeks a model that is both more robust across domains and more reliable in practical deployment.

In practice, the solution θ^* is obtained by optimizing differentiable loss functions defined over the domain-level losses $\mathcal{L}_i(\theta), \forall i \in \mathcal{D}$. Section 4 presents several such optimization strategies, including worst-k optimization, Group DRO, and uniform weighting.

4 ROBUST DOMAIN-AWARE ENTITY MATCHING (RODO-EM)

We first introduce a family of domain-level optimization objectives and their corresponding loss functions (Section 4.1). We then introduce a novel adaptive weighting mechanism that dynamically adjusts robustness based on domain-level training losses (Section 4.2).

Next, we present domain-aware sampling strategies that further encourage robust performance during training (Section 4.3).

Finally, Section 4.4 introduces RoDo-EM (**Ro**bust **Do**main-Aware **E**ntity **M**atching optimizer), a novel, unified framework that integrates adaptive robustness weighting, domain-aware sampling, and regularization techniques to improve performance on underperforming domains while maintaining strong matching performance.

4.1 Domain-Level Optimization Objectives

Although REMAD formally defines robustness as maximizing the performance of the worst-performing domain, the most appropriate domain-level objective ultimately depends on the requirements of a particular application. Accordingly, we generalize robustness to two major categories of domain-level optimization objectives: *performance-based* objectives and *parity-based* objectives. Performance-based objectives emphasize the predictive performance of a particular subset of domains, whereas parity-based objectives seek to reduce disparities in performance across all domains. Although these objective families are implemented differently, both seek to produce a robust EM model under a particular domain-level criterion.

For each optimization objective, we first define a target evaluation metric $m : \Theta \to \mathbb{R}$ that specifies the desired domain-level property (e.g., worst-domain F1 or F1 entropy). We then define a corresponding differentiable training loss $\mathcal{L} : \Theta \to \mathbb{R}$ that serves as a surrogate objective for optimizing this metric during gradient-based training. Because many evaluation metrics used throughout this paper (e.g., F1, precision, recall, and the associated parity metrics) depend on thresholded predictions and are therefore non-differentiable, the corresponding loss functions optimize differentiable approximations rather than the metrics themselves. For each domain $i \in \mathcal{D}$, we assume access to the domain-level loss $\mathcal{L}_i(\theta)$ and differentiable soft counterparts to F1, precision, and recall, denoted by $\tilde{F}_i(\theta)$, $\tilde{P}_i(\theta)$, and $\tilde{R}_i(\theta)$, respectively. These surrogate metrics are computed by replacing thresholded predictions with the model's predicted positive-class probabilities, yielding differentiable approximations that are used exclusively during optimization. For example, we estimate the soft true positives for domain i as

$$\tilde{TP}_i = \sum_{(x,y)\in\tilde{n}} p(x)y$$

where $p(x)$ denotes the model's predicted probability that sample x in the current minibatch $\tilde{n}$ belongs to the positive class. Soft precision, recall, and F1 are then defined analogously by replacing TP, FP, and FN with their soft counterparts. Once training is complete, these surrogate metrics are discarded and the original thresholded metrics are used for evaluation.

Performance-based Objectives. The most common objective in robust domain-aware optimization is to improve the k worst-performing domains, i.e., the domains with the lowest F1 scores. This objective encourages the model to allocate additional optimization effort toward the domains it currently finds most difficult, thereby improving worst-case robustness. In Example 1.1, we motivated the Worst-1 objective using a toy e-commerce dataset in which a minority domain of high-priced products contains more challenging matching relationships and incurs greater business costs when matching errors occur. We define the worst-k metric as the average F1 score over these domains:

$$m_{\text{worst-}k}(\theta) = \frac{1}{k} \sum_{i\in \text{BottomK}_{i\in\mathcal{D}}(F_i(\theta))} F_i(\theta). \tag{1}$$

The objective is to maximize $m_{\text{worst-}k}(\theta)$. This formulation can also extend to the best and median-performing domains by replacing BottomK with the corresponding domain-selection operator.

To optimize the Worst-k objective, we define a weighted loss that places additional emphasis on the selected domains:

$$\begin{aligned} \mathcal{S}_{\text{worst-}k} &= \text{TopK}_{i\in\mathcal{D}}(\mathcal{L}_i(\theta)), \\ \mathcal{L}_{\text{worst-}k}(\theta) &= \sum_{i\in\mathcal{S}_{\text{worst-}k}} \gamma_{\text{worst-}k}\mathcal{L}_i(\theta) + \sum_{i\notin\mathcal{S}_{\text{worst-}k}} \mathcal{L}_i(\theta), \end{aligned} \tag{2}$$

where $\gamma_{\text{worst-}k} > 1$ controls the additional emphasis assigned to the objective domains relative to the remaining domains. Consequently, $\mathcal{L}_{\text{worst-}k}$ increases the optimization effort devoted to the current domains with the highest loss while still allowing all other domains to contribute to the training objective.

We additionally consider a loss inspired by Group Distributionally Robust Optimization (Group DRO) [6, 48]. Distributionally Robust Optimization minimizes the worst-case expected loss over an uncertainty set of data distributions, while Group DRO instantiates these distributions as predefined groups, which naturally correspond to domains in our setting.

Unlike the formulation of Sagawa *et al.* [48], which explicitly maintains adversarial group weights that increasingly concentrate on the highest-loss groups throughout training, we employ a smooth log-sum-exp objective that emphasizes higher-loss domains while retaining contributions from all domains. The resulting loss is

$$\mathcal{L}_{\text{DRO}}(\theta) = \frac{1}{\gamma_{\text{DRO}}} \log\left(\sum_{i\in\mathcal{D}} \exp(\gamma_{\text{DRO}}\mathcal{L}_i(\theta))\right), \tag{3}$$

where $\gamma_{\text{DRO}} > 0$ controls the degree of robustness. As γ_{DRO} increases, the objective places progressively greater emphasis on higher-loss domains. Smaller values approach a uniformly weighted objective.

Parity-based Objectives. Parity-based objectives seek to reduce disparities across domains rather than emphasizing a particular subset of domains. Because factors such as domain difficulty and training-set size can naturally produce uneven performance, a practitioner may prefer an objective that treats every domain equally.

We define a *uniform* objective that maximizes the macro-average domain F1 score:

$$m_{\text{uniform}}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{i\in\mathcal{D}} F_i(\theta). \tag{4}$$

The corresponding training loss assigns equal weights:

$$\mathcal{L}_{\text{uniform}}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{i\in\mathcal{D}} \mathcal{L}_i(\theta). \tag{5}$$

Unlike the standard sample-level cross-entropy objective, which implicitly gives greater influence to domains containing more training samples, the uniform loss gives every domain equal importance. Consequently, the training objective becomes a macro-average over domains rather than a micro-average over individual samples.

Maximizing average domain performance alone, however, does not guarantee that performance is evenly distributed. A model may

achieve a high macro-average F1 score while still containing one or more domains whose performance differs substantially from the others. We therefore define four additional parity metrics that can be optimized jointly with average predictive performance: normalized F1 entropy, F1 variance, Positive Predictive Value Parity (PPVP), and True Positive Rate Parity (TPRP). PPVP and TPRP correspond to parity in domain-level precision and recall, respectively, following prior work on fairness in EM [51].

For F1 entropy, we first normalize the domain-level F1 scores:

$$q_i(\theta) = \frac{F_i(\theta)}{\sum_{z\in\mathcal{D}} F_z(\theta)}. \tag{6}$$

We then define the normalized F1 entropy as

$$m_{\text{entropy}}(\theta) = -\frac{1}{\log|\mathcal{D}|}\sum_{i\in\mathcal{D}} q_i(\theta)\log q_i(\theta). \tag{7}$$

The normalization constrains $m_{\text{entropy}}(\theta)$ to $[0,1]$, with larger values indicating that F1 performance is more evenly distributed across domains. Let

$$\overline{F}(\theta) = \frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}} F_i(\theta), \quad \overline{P}(\theta) = \frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}} P_i(\theta), \text{and}$$

$$\overline{R}(\theta) = \frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}} R_i(\theta)$$

denote the corresponding domain-level means. We define the remaining parity metrics as the negative variance of the relevant performance measure:

$$m_{\text{var}}(\theta) = -\frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\left(F_i(\theta)-\overline{F}(\theta)\right)^2, \tag{8}$$

$$m_{\text{PPVP}}(\theta) = -\frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\left(P_i(\theta)-\overline{P}(\theta)\right)^2, \tag{9}$$

$$m_{\text{TPRP}}(\theta) = -\frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\left(R_i(\theta)-\overline{R}(\theta)\right)^2. \tag{10}$$

Because these metrics are defined as negative variances, values closer to zero indicate greater parity. Defining all four parity measures as higher-is-better metrics allows them to be incorporated consistently into the training objective. However, in Section 6 we report the positive counterparts for consistency.

For each parity metric, we define a differentiable surrogate by replacing the thresholded domain-level F1, precision, and recall values with their corresponding soft counterparts. Consequently, the surrogate metrics have the same functional form as Equations (7)–(10), differing only in that F_i, P_i, and R_i are replaced by $\tilde{F}_i$, $\tilde{P}_i$, and $\tilde{R}_i$, respectively. For example, let $\overline{\tilde{F}}(\theta) = \frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\tilde{F}_i(\theta)$.

The differentiable surrogate for F1 variance is then defined as

$$\tilde{m}_{\text{var}}(\theta) = -\frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\left(\tilde{F}_i(\theta)-\overline{\tilde{F}}(\theta)\right)^2. \tag{11}$$

We then optimize the following parity-regularized loss for a given metric "par":

$$\mathcal{L}_{\text{par}}\left(\theta;\tilde{m}_{\text{par}}\right) = \mathcal{L}(\theta) - \gamma_{\text{par}}\tilde{m}_{\text{par}}(\theta), \tag{12}$$

where $\gamma_{\text{par}} > 0$ controls the trade-off between overall predictive performance and domain parity. Larger values place greater emphasis on improving domain parity, whereas smaller values prioritize minimizing the average prediction loss. By jointly optimizing prediction loss and parity, this formulation encourages more uniform performance across domains while maintaining overall accuracy.

4.2 Adaptive Robustness Weighting

Most of the optimization objectives introduced in Section 4.1, with the exception of $\mathcal{L}_{\text{uniform}}$, rely on a *robustness weight* that controls the strength of the corresponding robustness objective (e.g., $\gamma_{\text{worst-}k}$, γ_{DRO}, or γ_{par}). Although these weights can be treated as additional hyperparameters, doing so places an additional burden on the practitioner and requires selecting an appropriate robustness strength for each optimization configuration.

Accordingly, we introduce a novel *adaptive weighting scheme* in which the robustness strength is determined dynamically during training from the distribution of domain-level losses. Let $\mathcal{L}_i^{(t)}$ denote the loss of domain $i \in \mathcal{D}$ after epoch t. To determine the robustness weight for epoch t, we first compute the mean and standard deviation of the domain losses from the preceding epoch:

$$\mu^{(t-1)} = \frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\mathcal{L}_i^{(t-1)}, \qquad \sigma^{(t-1)} = \sqrt{\frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\left(\mathcal{L}_i^{(t-1)}-\mu^{(t-1)}\right)^2}. \tag{13}$$

We then identify the highest-loss domain from the preceding epoch:

$$d_{t-1}^* = \arg\max_{i\in\mathcal{D}} \mathcal{L}_i^{(t-1)}. \tag{14}$$

Its standardized deviation from the mean is defined as

$$z^{(t-1)} = \frac{\mathcal{L}_{d_{t-1}^*}^{(t-1)} - \mu^{(t-1)}}{\sigma^{(t-1)}+\epsilon}, \tag{15}$$

where $\epsilon > 0$ is a small constant included for numerical stability. The adaptive robustness score for epoch t is then

$$s_t = \left(z^{(t-1)}\right)^2 = \left(\frac{\max_{i\in\mathcal{D}}\mathcal{L}_i^{(t-1)} - \mu^{(t-1)}}{\sigma^{(t-1)}+\epsilon}\right)^2. \tag{16}$$

The score s_t is then mapped to the robustness weight required by the corresponding objective. For objectives whose robustness weight must exceed one, we adjust the score accordingly. In particular, for the Worst-k objective we define $\gamma_{\text{worst-}k,t} = 1 + s_t$, which ensures that the selected domains always receive greater weight than the remaining domains. For the DRO objective, we similarly use $\gamma_{\text{DRO},t} = \max(1, s_t)$, while the parity-based objectives use s_t directly as their adaptive robustness weight.

The resulting epoch-specific robustness objective can be expressed generally as

$$\mathcal{L}_{\text{robust}}^{(t)} = \mathcal{L}_{\text{robust}}\left(\{\mathcal{L}_i^{(t)}\}_{i\in\mathcal{D}};\gamma_t\right), \tag{17}$$

where γ_t denotes the objective-specific adaptive robustness weight derived from s_t.

For example, under the DRO objective, the adaptive weight $\gamma_{\text{DRO},t}$ replaces the fixed robustness-strength parameter γ_{DRO}:

$$\mathcal{L}_{\text{DRO}}^{(t)} = \frac{1}{\gamma_{\text{DRO},t}} \log \sum_{i \in \mathcal{D}} \exp\left(\gamma_{\text{DRO},t} \mathcal{L}_i^{(t)}\right). \tag{18}$$

Thus, the robustness strength is determined dynamically from the cross-domain training-loss distribution rather than selected as a fixed hyperparameter. When the highest-loss domain deviates more strongly from the remaining domains, the resulting adaptive score increases, producing greater robustness pressure on underperforming domains during the subsequent epoch.

The adaptive formulation can be generalized by computing domain statistics over a sliding window of previous epochs, recomputing the weight at finer granularities such as training iterations, or varying the frequency at which the weight is updated. In the main experiments, we adopt the preceding-epoch formulation described above. We provide an ablation of these alternative adaptive configurations in the accompanying technical report.

4.3 Domain-Aware Batch Sampling

As an additional component of the REMAD optimization framework, we consider how the composition of each training batch can be used to further emphasize the training objectives introduced in Section 4.1. While the loss functions we defined determine how strongly each domain contributes to the optimization objective, the sampling strategy determines how frequently samples from each domain are presented to the model during training.

Consider a mini-batch constructed at training iteration e. We define the batch sampling distribution by a probability distribution P_{opt} over the domains, where $P_{\text{opt}}(i)$ denotes the probability that a sample selected for the mini-batch is drawn with replacement from domain i under sampling strategy *opt*. Different sampling strategies therefore correspond to different choices of P_{opt}, allowing the training process to emphasize different subsets of domains.

Balanced Batches. A simple strategy is to enforce equal representation from every domain within each mini-batch. To achieve this, domains are sampled with uniform probability:

$$P_{\text{BAL},e}(i) = \frac{1}{|\mathcal{D}|}.$$

Consequently, each domain contributes approximately the same number of samples to every mini-batch. This strategy prevents large domains from dominating individual optimization steps and naturally complements the uniform weighting objective.

Sampling by Loss. Rather than sampling every domain equally, the sampling distribution can adapt dynamically according to the current performance of each domain. Let $\widehat{\mathcal{L}}_i(\theta_e)$ and $F_i(\theta_e)$ denote the exponentially smoothed domain-level loss and domain-level F1 score, respectively, after epoch e.

When sampling according to loss, each training sample from domain i is assigned a weight $\widehat{\mathcal{L}}_i(\theta_e)$. Consequently, the probability of selecting samples from domain i is

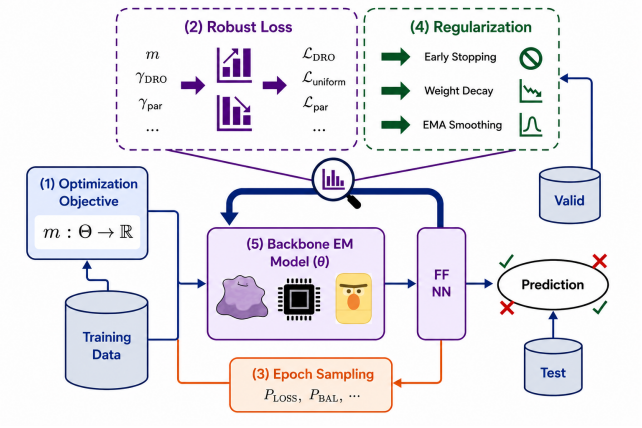


Figure 3: Overview of the RoDo-EM framework.

$$P_{\text{Loss},e}(i) = \frac{|n_i| \widehat{\mathcal{L}}_i(\theta_e)}{\sum_{k \in \mathcal{D}} |n_k| \widehat{\mathcal{L}}_k(\theta_e)},$$

where $|n_i|$ denotes the number of training samples in domain i. Thus, domains with larger losses receive proportionally more samples during subsequent training iterations, although their total sampling probability also depends on domain size.

Similarly, sampling can be based on domain-level F1 scores by assigning a weight of $1 - F_i(\theta_e)$ to each training sample from domain i, giving greater probability to domains with lower performance:

$$P_{\text{F1},e}(i) = \frac{|n_i| \left(1 - F_i(\theta_e)\right)}{\sum_{k \in \mathcal{D}} |n_k| \left(1 - F_k(\theta_e)\right)}.$$

Unlike balanced sampling, these adaptive sampling strategies evolve throughout training, allowing the model to allocate additional optimization effort toward domains that are currently performing poorly.

4.4 RoDo-EM

Having introduced the optimization objectives and domain-aware sampling strategies used throughout this work, we now present RoDo-EM (**Ro**bust **Do**main-Aware **E**ntity **M**atching optimizer), our unified framework for robust optimization in domain-aware EM.

Definition 2. RoDo-EM is a configurable optimization framework that combines a backbone EM model with modular robustness components to produce a robust domain-aware EM training pipeline. A RoDo-EM configuration is formally defined as

$$\texttt{RoDo-EM} = (\mathcal{O}, \mathcal{L}, \mathcal{P}, \mathcal{R}, \mathcal{M}),$$

where $\mathcal{O}$ denotes the optimization objective, $\mathcal{L}$ the robust loss function, $\mathcal{P}$ the domain-aware sampling strategy, $\mathcal{R}$ the set of chosen regularization strategies, and $\mathcal{M}$ the backbone EM model.

Figure 3 illustrates the RoDo-EM framework. Training begins by specifying (1) a domain-level optimization objective, which is controlled by a metric m according to the desired notion of robustness (e.g., worst-performing, F1 variance, PPVP variance, etc.). This objective defines (2) a corresponding robust loss function, such as the weighted worst-k, Group DRO, or uniform objective introduced

Table 1: Dataset characteristics for the three EM benchmarks.

Dataset	# Samples	Pos. (%)	# Domains	Avg. Domain Size	# Attr.	# Entities
WDC-MD	21,162	43.63	12	1,764	1	4,332
WDC4	13,003	26.15	4	3,251	1	10,651
Abt-Buy	5,781	9.20	10	578	3	1,103

in Section 4.1. During optimization, (3) a domain-aware sampling strategy from Section 4.3 determines how training samples are selected for each mini-batch according to the chosen objective (e.g., balanced sampling, loss-based sampling, or F1-based sampling).

These optimization components operate jointly with (4) standard regularization techniques and (5) a backbone EM model (e.g. fine-tuned with DITTO [28]). Specifically, we incorporate early stopping based on validation performance, L2 weight decay to discourage overly complex models, and Exponential Moving Average (EMA) smoothing to stabilize optimization. Together, these regularization techniques improve generalization and complement the robust optimization objectives. The resulting training procedure can be applied to any supervised EM backbone, such as a PLM or a fine-tuned LLM. After training, the best-performing model is selected based on validation set performance and evaluated on the domain-partitioned test data.

A key characteristic of RoDo-EM is its modularity. In practice, applications differ in their robustness requirements, computational constraints, and available backbone models. RoDo-EM is thus designed to allow each component of the optimization pipeline to be selected independently. Different optimization objectives, robust loss functions, sampling strategies, regularization methods, and backbone models can be combined to accommodate diverse application requirements without modifying the overall framework.

Accordingly, in our experimental evaluation, robustness weights (e.g., $\gamma_{\texttt{worst-k}}$, $\gamma_{\texttt{DRO}}$, and $\gamma_{\texttt{par}}$) are determined adaptively during training using the method from Section 4.2. The remaining configuration choices, including batch sampling strategies and regularization hyperparameters (e.g., EMA smoothing and L2 weight decay), are selected independently through grid search on the validation set.

5 EXPERIMENTAL SETUP

We now turn to a comprehensive experimental evaluation to assess the effectiveness of the proposed robust domain-aware optimization techniques. Section 5.1 introduces the benchmark datasets and domain partitions used throughout the evaluation. Section 5.2 describes the implementation details, and Section 5.3 presents the baseline methods against which we compare our approach.

5.1 Datasets

We evaluate the REMAD setting on three widely used product EM benchmarks: the WDC Multi-Dimensional Entity Matching Benchmark (WDC-MD) [42], the WDC Products Corpus [44], and Abt-Buy [27]. Product-focused benchmarks cover a diverse range of domains, making them particularly well suited for this setting. Dataset characteristics for the benchmarks are provided in Table 1.

Table 2: WDC-MD domain-level sample distributions across training, validation, and test splits. Test F1 is reported for the baseline fine-tuned DistilBERT model.

	Train		Validation		Test		
Domain	Samples	Pos. (%)	Samples	Pos. (%)	Samples	Pos. (%)	F1
Computers	6,473	51.2	1,174	14.6	790	15.6	0.815
Electronics	2,759	53.5	483	17.0	391	18.0	0.762
Office	1,800	47.2	467	16.4	496	16.5	0.805
Sports	1,715	59.4	256	18.1	211	19.9	0.736
Cameras	1,317	57.7	204	17.7	172	18.8	0.950
Jewelry	526	42.4	183	17.8	318	17.0	0.784
Instruments	266	53.4	54	20.4	100	19.1	0.933
Clothing	219	58.2	28	21.2	37	23.3	1.000
Cell Phones	151	69.5	24	36.8	48	24.5	0.933
Tools	129	77.9	14	41.4	29	28.7	0.889
Automotive	120	57.4	32	23.7	23	26.0	0.421
Home/Garden	60	47.5	25	23.9	125	20.9	0.750

Domain Partitioning. Following [45], we partition each EM benchmark into domains based on a semantically meaningful attribute. The choice of domain partition is important because RoDo-EM explicitly optimizes performance across these groups. Accordingly, we prioritize attributes that capture meaningful differences in matching characteristics rather than arbitrary subdivisions of the data. In our e-commerce setting, product categories and brands provide natural partitions, as they can exhibit distinct attribute distributions, naming conventions, and product specifications relevant to identifying matching entities. For example, matching computer products may require distinguishing technical specifications, whereas matching clothing products may rely more heavily on brand, size, and style information. We use the category labels provided by the WDC-MD and WDC Products Corpus benchmarks and construct brand-based partitions for Abt-Buy. When necessary, we retain only domains containing at least 100 total samples to ensure sufficient data for domain-aware training and evaluation. Collectively, these datasets exhibit diverse domain distributions, allowing us to evaluate REMAD under a variety of scenarios.

Table 2 characterizes the product-category partition of WDC-MD, reporting domain sizes, positive (match) ratios, and baseline test F1 scores. The substantial variation in domain sizes and class distributions illustrates the heterogeneity of the benchmark, while differences in baseline F1 indicate that matching difficulty also varies across domains. Notably, domain size alone does not determine matching difficulty. Some smaller domains achieve high F1 scores while others perform much worse. We preserve the benchmark's original training, validation, and test splits without rebalancing their domain distributions, providing a realistic setting for evaluating robustness across heterogeneous domains.

The WDC Multi-Dimensional Benchmark (WDC-MD). WDC-MD [42] is a large product matching benchmark containing datasets that vary along three dimensions: the percentage of corner cases, the percentage of unseen entities in the test set, and the dataset size. Throughout this paper, we use the large variant with the 50% corner-case and 50% unseen-entity setting, providing sufficient samples for each domain while representing the benchmark's central evaluation configuration.

We partition WDC-MD using its provided *product category* labels. This produces a right-skewed distribution across 12 domains,

with several minority categories (e.g., "Automotive" and "Musical Instruments") and a few dominant categories (e.g., "Computers" and "Office Products"). Consequently, WDC-MD provides a challenging REMAD setting in which minority domains are particularly susceptible to being overshadowed during training, making it well suited for evaluating robust optimization techniques.

The WDC Products Corpus. We also include the medium variant of the WDC Products Corpus [44], a separate product-based benchmark from WDC with a more balanced distribution than WDC-MD. Similar to WDC-MD, this benchmark is partitioned by product category. The corpus contains four categories–"Cameras," "Computers," "Shoes," and "Watches"–whose domain sizes range from approximately 4k to 6k samples. Compared to WDC, these domains are considerably more balanced, allowing us to evaluate REMAD when all domains are well represented in the training data.

The Abt-Buy Benchmark. The Abt-Buy benchmark [27] is a widely used EM benchmark that aligns product listings from abt.com with listings from buy.com. Compared to the WDC datasets, Abt-Buy contains a more text-heavy and low-attribute schema, allowing us to evaluate REMAD in settings where domain boundaries arise from heterogeneous textual representations rather than standardized attributes. We partition Abt-Buy by *brand*, producing 10 domains with a similarly right-skewed distribution.

5.2 Implementation Details

All experiments were conducted on NVIDIA A100 (80GB) GPUs within a high-performance computing cluster running Linux and Python 3.7.[2] Unless otherwise stated, all experiments fine-tune **DistilBERT** [49] as the PLM backbone. We adopt the dataset-specific hyperparameter settings from DITTO [28]. Regularization hyperparameters (η, ω) are selected independently for each optimization method using a grid search.

We evaluate RoDo-EM using several domain-level metrics corresponding to the optimization objectives introduced in Section 4.1. Let $F_i(\theta)$ denote the F1 score for domain $i \in \mathcal{D}$. We report weighted F1, which averages domain-level F1 scores according to their test-set sizes, and macro F1, which assigns equal weight to each domain. To assess robustness, we additionally report minimum-k F1, defined as the average F1 of the k lowest-performing domains, along with parity metrics including F1 entropy, F1 variance, Positive Predictive Value Parity (PPVP), and True Positive Rate Parity (TPRP). Formal definitions and their corresponding optimization objectives are summarized in the accompanying technical report.

We additionally measure the runtime overhead of robust optimization using DistilBERT. Across our three benchmarks, RoDo-EM increases training time by approximately 6–12% compared to standard fine-tuning, while inference latency remains effectively unchanged at approximately 0.7–0.8 ms per candidate pair. These results suggest that domain-aware robust optimization introduces modest computational overhead without additional inference cost.

5.3 Baselines

To evaluate the effectiveness of RoDo-EM, we compare against several recent EM methods spanning general-purpose, domain-specific, graph-based, budget-aware, and prompt-based approaches. Unless otherwise stated, all PLM-based methods use the same backbone model, hyperparameters, and training data as RoDo-EM to ensure a fair comparison.

GEN (General EM). GEN is the standard general-purpose EM model obtained by fine-tuning a PLM on the union of the training data from all domains. It serves as the primary baseline for evaluating whether robust optimization improves the performance of a single model trained across multiple domains.

SPEC (Domain-Specific EM). SPEC trains one PLM for each domain using only the samples belonging to that domain. While this allows each model to specialize to its corresponding domain, it requires maintaining multiple models and may underperform when individual domains contain limited labeled data.

FlexER [21]. FlexER is a recent PLM-based EM framework that combines transformer representations with a Graph Neural Network (GNN) to support multiple matching intents. For our evaluation, we train FlexER using two intents: (1) the standard EM labels and (2) a coarse-grained product-type intent that is distinct from the domain partitions used in REMAD. This provides a strong graph-based baseline for evaluating domain-aware EM.

BEACON [45, 46]. BEACON is a recent framework for EM across domains that proposes distribution-aware sampling strategies to improve domain-level performance under a fixed labeling budget. While the original BEACON framework trains domain-specific EM models, a recently introduced general-purpose (i.e., domain-agnostic) variant [46] adapts these sampling strategies to

Table 3: Robustness metrics for RoDo-EM compared with other modern EM methods over three benchmark datasets.

(a) WDC-MD (50% cc, 50% unseen)

Model	Min. ↑	Macro ↑	Weighted ↑	F1 Entropy ↑	PPVP Var. ↓
GEN (Ditto)	0.421	0.815	**0.810**	0.993	0.026
SPEC (Ditto)	0.240	0.506	0.616	0.978	0.052
FlexER	0.491	0.656	0.658	0.993	0.024
BEACON (WS)	0.533	0.688	0.702	0.996	0.023
Llama	0.430	0.659	0.595	0.991	0.032
Jellyfish	0.423	0.692	0.691	0.992	0.016
RoDo-EM	**0.714**	**0.854**	**0.810**	**0.997**	**0.008**

(b) WDC Product Corpus

Model	Min. ↑	Macro ↑	Weighted ↑	F1 Entropy ↑	PPVP Var. ↓
GEN (Ditto)	0.814	0.864	**0.874**	0.999	0.001
SPEC (Ditto)	0.780	0.848	0.847	0.999	0.005
FlexER	0.774	0.834	0.834	0.999	0.002
BEACON (WS)	0.812	0.868	0.868	0.999	0.002
Llama	0.711	0.766	0.767	0.999	0.001
Jellyfish	0.789	0.836	0.837	0.999	0.003
RoDo-EM	**0.857**	**0.869**	**0.874**	**1.000**	**0.000**

(c) Abt-Buy

Model	Min. ↑	Macro ↑	Weighted ↑	F1 Entropy ↑	PPVP Var. ↓
GEN (Ditto)	0.333	0.796	**0.810**	0.993	0.043
SPEC (Ditto)	0.000	0.163	0.252	0.808	0.028
FlexER	0.000	0.626	0.688	0.941	0.096
BEACON (WS)	0.364	0.732	0.772	0.988	0.061
Llama	0.167	0.388	0.366	0.967	0.020
Jellyfish	0.500	0.793	0.797	0.991	0.011
RoDo-EM	**0.750**	**0.884**	**0.810**	**0.997**	**0.003**

[2] See https://anonymous.4open.science/r/RODO-EM-8350 for open-source code, data, and a technical report.

train a single EM model across all domains. We use this general-purpose variant as the primary baseline for comparison with RoDo-EM, employing the default 70% downsampling rate from the original work. We use BEACON as our representative domain-aware baseline, as recent evaluations demonstrated that it surpasses MFSN [53], a method previously shown to outperform DADER [56].

We additionally compare RoDo-EM against several prompt-based EM methods, including those built upon LLMs. Following Peeters *et al.* [43], we adopt the *general-complex* prompting strategy to evaluate two representative methods:

Llama 3.1 [15], an open-source general-purpose LLM with approximately 8B parameters.

Jellyfish [68], an LLM fine-tuned for data management tasks, including EM. For our results, we use the Jellyfish-8b model.

6 RESULTS AND ANALYSIS

In this section, we evaluate the RoDo-EM framework and analyze its performance on the REMAD problem setting across multiple domain-partitioned datasets and robustness objectives. Section 6.1 presents the primary experimental results, and Section 6.2 performs various ablation studies on various components of RoDo-EM.

6.1 Main Results

We begin by evaluating RoDo-EM across the five robustness metrics introduced in Table 8 and comparing its performance against several modern EM baselines. The reported metrics include minimum domain F1, macro F1, weighted F1, F1 entropy, and PPVP variance. For each metric, we configure RoDo-EM to optimize the corresponding objective by selecting the appropriate loss function (e.g., $\mathcal{L}_{\text{DRO}}$ for min-F1, $\mathcal{L}_{\text{uniform}}$ for macro F1, etc.), implement the objective with adaptive weighting, and perform small grid search over the associated sampling strategy and regularization hyperparameters. We then evaluate how effectively each objective improves its target metric while influencing the remaining evaluation metrics.

Raw Performance. Table 3 summarizes the raw performance of all methods on WDC-MD, the WDC Product Corpus, and Abt-Buy. Across all datasets, RoDo-EM achieves the strongest performance in each of the evaluation metrics. These improvements stem from jointly optimizing robust loss functions, employing batch sampling techniques to improve domain exposure during training, and incorporating the regularization techniques described in Section 4.4. Although the magnitude of improvement varies across datasets, consistent gains are observed for each objective. It should be noted that the GEN (Ditto) baseline and the weighted-F1 variant of RoDo-EM achieve identical weighted F1 scores for each dataset because optimizing weighted F1 corresponds to the standard training objective, $\mathcal{L}(\theta)$, used by the underlying DITTO model.

On WDC-MD (Table 3a), RoDo-EM achieves a substantial improvement in minimum domain F1, outperforming the next-best baseline by 34.0%. Improvements in the remaining robustness metrics are more modest but still significant, including a 4.8% increase in macro F1. Domain parity also improves, with PPVP variance reduced from 0.016 to 0.008 compared to the next-best baseline, accompanied by a smaller improvement in F1 entropy. **These results suggest that RoDo-EM is particularly effective when training data exhibit substantial domain imbalance.**

Table 3b shows more modest improvements across all evaluation metrics. Optimizing minimum domain F1 yields a 5.3% improvement over the next-best baseline. Because the WDC Product Corpus contains a more uniformly distributed set of training data across domains, both F1 entropy and PPVP variance exhibit only marginal improvements, as parity is already high prior to optimization. The macro F1 score obtained by $\mathcal{L}_{\text{uniform}}$ is nearly identical to that achieved by the BEACON baseline, suggesting that the advantages of RoDo-EM are more apparent when domain imbalance is present.

Finally, Table 3c demonstrates that the benefits of RoDo-EM are again substantial on Abt-Buy. Min. domain F1 improves by 50.0% over the next-best baseline, while macro F1 increases by 11.1%. Although F1 entropy improves only slightly compared to the base GEN model, RoDo-EM noticeably reduces PPVP variance from 0.043 to 0.003. The textual nature of Abt-Buy enables the Jellyfish LLM to outperform many of the remaining baselines; however, incorporating robustness-aware optimization within RoDo-EM produces further improvements in both robustness and domain parity.

Optimization Objectives. Table 4 presents an ablation study of each optimization objective within the RoDo-EM framework. For each objective, we report the improvement in its target metric, the corresponding changes in the remaining robustness metrics (with target improvement % italicized), and the percentage of domains whose performance either improves or remains unchanged. This analysis characterizes the tradeoffs associated with optimizing different notions of robustness. Italicized values denote the relative improvement achieved in the objective's target metric.

The results show that optimizing for the worst-performing domains yields substantially larger gains in the target metric than the accompanying degradation in average-case performance. For example, $\mathcal{L}_{\text{worst-1}}$ and $\mathcal{L}_{\text{DRO}}$ improve the minimum domain F1 score by 70.5% and 69.7%, respectively, while reducing weighted F1 by only 4.5% and 4.7%. Interestingly, both objectives also improve the domain parity metrics as a secondary effect. This behavior is expected, as raising the performance of the weakest domains while maintaining comparable performance on the remaining domains naturally produces a more balanced distribution of performance across domains. Although weighted F1 decreases modestly relative to the substantial gains in minimum domain performance, this tradeoff remains an important consideration for practitioners when selecting an optimization objective.

The parity-based objectives in Table 4 exhibit a similar tradeoff. Optimizing F1 entropy, F1 variance, PPVP variance, or TPRP variance decreases weighted F1 by roughly 4-8% while producing improvements in the corresponding target metric. Furthermore, optimizing PPVP variance and TPRP variance reduced weighted F1 by 7.1% and 4.1%, respectively. Intuitively, encouraging greater parity across domains often requires sacrificing some performance on stronger domains in order to improve weaker ones. Consistent with this interpretation, all four parity objectives also increase the minimum domain F1 score by approximately 25-50%, indicating that improving parity generally benefits the lowest-performing domains. Interestingly, the objective explicitly optimizing a given parity metric does not always produce the largest improvement in that metric, although the differences are generally small. For example, $\mathcal{L}_{\text{DRO}}$ and $\mathcal{L}_{\text{par}}(\theta; m_{\text{var}})$ slightly outperform the F1 entropy objective on F1 entropy. This suggests that the robustness objectives–especially

Table 4: Ablation of RoDo-EM optimization objectives on WDC-MD.

Method	Target Objective	Target Metric		Relative Improvement (%) Across Metrics							
		Base	Robust	Worst F1	Macro F1	Weighted F1	F1 Entropy	F1 Var. Reduction	PPVP Var. Reduction	TPRP Var. Reduction	% Domains Improved
Worst-1	Minimum Domain F1	0.421	**0.718**	*+70.5*	+2.0	-4.5	+0.5	+60.2	+57.1	+46.7	50.0% (6/12)
DRO	Minimum Domain F1	0.421	**0.714**	*+69.7*	+0.9	-4.7	+0.4	+58.6	+42.2	+38.6	50.0% (6/12)
Uniform	Macro F1	0.815	**0.854**	+58.3	*+4.8*	+0.0	+0.4	+52.2	+59.2	+45.2	75% (9/12)
Entropy	F1 entropy	0.993	**0.997**	+63.2	+0.5	-6.8	*+0.4*	+61.0	+50.7	+59.7	58.3% (7/12)
F1 Variance	F1 variance	0.022	**0.008**	+67.7	+3.0	-2.7	+0.5	*+62.7*	+63.3	+40.8	58.3% (7/12)
PPVP	PPVP variance	0.026	**0.008**	+66.1	+3.5	-7.1	+0.4	+47.4	*+68.0*	+10.1	50.0% (6/12)
TPRP	TPRP variance	0.022	**0.007**	+58.4	+1.5	-4.1	+0.4	+58.8	+47.9	*+66.0*	41.7% (5/12)

Table 5: RoDo-EM ablation on WDC-MD with $\mathcal{L}_{\text{DRO}}$ objective.

Configuration	Worst F1	Macro F1	Weighted F1
Base Model	0.421	0.815	**0.806**
Weighting Only	0.533	0.773	0.732
Batch Sampling Only	0.631	0.790	0.680
Full RoDo-EM	**0.714**	**0.822**	0.772

Table 6: RoDo-EM performance comparison for DistilBERT and Llama across optimization objectives on WDC-MD.

		DistilBERT			Llama-3.1-8B		
Optimizer	Target Metric	Base	RoDo-EM (Δ)	Weighted F1 (Δ)	Base	RoDo-EM (Δ)	Weighted F1 (Δ)
Worst-1	Min. F1 ↑	0.421	0.718 (+0.297)	0.773 (−0.037)	0.706	0.764 (+0.058)	0.871 (−0.008)
DRO	Min. F1 ↑	0.421	0.714 (+0.293)	0.772 (−0.038)	0.706	0.771 (+0.065)	0.877 (−0.002)
Uniform	Macro F1 ↑	0.815	0.854 (+0.039)	0.810 (+0.000)	0.907	0.909 (+0.002)	0.880 (+0.001)
F1 Variance	F1 Var. ↓	0.022	0.008 (−0.014)	0.788 (−0.022)	0.009	0.006 (−0.003)	0.876 (−0.003)

based on parity–are closely related, with optimizing one objective often yielding comparable improvements across related metrics.

Among the evaluated objectives, $\mathcal{L}_{\text{uniform}}$ exhibits the fewest apparent tradeoffs. Although its target improvement is more modest, increasing macro F1 by 4.8%, it does not degrade any of the remaining evaluation metrics. Moreover, it achieves the highest percentage of improved domains, with 75% (9 of 12) exhibiting improved performance. This behavior closely aligns with the design of $\mathcal{L}_{\text{uniform}}$, which assigns equal importance to every domain during optimization. Overall, these results demonstrate that different robustness objectives provide practitioners with distinct and interpretable tradeoffs, allowing RoDo-EM to be tailored to the requirements of a particular application.

6.2 Ablation Study

We next conduct an ablation study to investigate three aspects of RoDo-EM: the individual contributions of robust weighting and batch sampling (Section 6.2.1), generalization to LLM backbones (Section 6.2.2), and the effect of varying the number of worst-performing domains prioritized during optimization (Section 6.2.3). We use WDC-MD throughout these experiments due to its relatively large number of domains and heterogeneous domain distribution.

6.2.1 Weighting and Sampling Component Ablation. We isolate the contributions of two primary components of RoDo-EM: adaptive robustness weighting and domain-aware batch sampling. Using the $\mathcal{L}_{\text{DRO}}$ objective on WDC-MD, we evaluate four configurations: (1) the baseline DistilBERT model without robust optimization, (2) adaptive robustness weighting alone, (3) balanced batch sampling alone, and (4) the full RoDo-EM configuration combining both components. We hold constant all other hyperparameter and regularization settings across these configurations. Table 5 reports the resulting worst-domain, macro, and weighted F1 scores.

Both components independently improve worst-domain F1 over the baseline (0.421), with balanced sampling (0.631) providing a larger improvement than adaptive weighting alone (0.533). However, both configurations reduce macro and weighted F1, with sampling alone incurring the largest reduction in weighted F1 (0.680). Combining the two components yields the highest worst-domain F1 (0.714) and macro F1 (0.822), while recovering weighted F1 to 0.772. Although the baseline retains the highest weighted F1 (0.806), the combined configuration achieves a more favorable tradeoff between robustness and overall performance than either isolated component. **The results suggest that adaptive robustness weighting and balanced sampling provide complementary benefits**. Their combination mitigating some of the aggregate performance degradation observed when either component is applied independently.

6.2.2 Robust Optimization for LLMs. We next investigate whether the robustness improvements introduced by RoDo-EM generalize across backbone language models. Specifically, we compare DistilBERT, the PLM backbone used throughout our main experiments, with an 8B-parameter Llama-3.1 LLM [15]. For each backbone, we evaluate four RoDo-EM objectives: $\mathcal{L}_{\text{worst-1}}$, $\mathcal{L}_{\text{DRO}}$, $\mathcal{L}_{\text{uniform}}$, and $\mathcal{L}_{\text{par}}(\theta; m_{\text{var}})$. We additionally conducted experiments with several other PLM backbones and observed similar trends. For conciseness, we report DistilBERT and Llama-3.1 as representative models spanning substantially different model scales.

For Llama-3.1, we make two modifications to accommodate its larger size. First, we fine-tune using Low-Rank Adaptation (LoRA) [24], rather than the full-model fine-tuning used for DistilBERT. Second, we use a reduced grid search over the RoDo-EM hyperparameters (e.g., γ) to limit computational cost. The purpose of this experiment is not to identify the strongest backbone, but to determine whether RoDo-EM remains effective as backbone capacity increases. We therefore retain DistilBERT in the main experiments to isolate the effects of robust optimization under a consistent and computationally efficient setting, while using Llama-3.1 here to evaluate the generality of the framework.

Table 6 shows that RoDo-EM improves the target metric for all four objectives with Llama-3.1. The performance-based objectives

produce the largest gains: $\mathcal{L}_{\text{worst-1}}$ and $\mathcal{L}_{\text{DRO}}$ improve minimum domain F1 by 0.058 and 0.065, respectively. For the parity-based objectives, $\mathcal{L}_{\text{uniform}}$ improves macro F1 by 0.002, while $\mathcal{L}_{\text{par}}(\theta; m_{\text{var}})$ reduces F1 variance from 0.009 to 0.006. Although these improvements are smaller in magnitude than those observed with DistilBERT, the Llama-3.1 baseline begins with stronger target-metric performance, leaving less room for improvement. At the same time, RoDo-EM incurs smaller weighted F1 reductions with Llama-3.1 than DistilBERT, indicating larger backbones optimizing for robustness may incur a lower aggregate performance trade-off.

Overall, **RoDo-EM remains effective as backbone capacity increases**. By replacing the DistilBERT backbone in our main results with Llama, RoDo-EM still improves each targeted robustness metric while largely preserving weighted F1. Together with the consistent trends observed across our additional backbone experiments, these results indicate that RoDo-EM is complementary to, rather than dependent on, the choice or scale of the language model.

6.2.3 Worst-k Robustness Optimization. In Section 4.1, we define the worst-domain objective $\mathcal{L}_{\text{worst-}k}$ for arbitrary values of k. This flexibility is useful when a practitioner wishes to prioritize several challenging or underrepresented domains rather than only the single worst-performing domain. We therefore evaluate how varying k affects both lower-bound and average-case performance.

WDC-MD yields $|\mathcal{D}| = 12$, and we evaluate seven values of k in the range $[1, 7]$. For each value, we utilize the adaptive weighting strategy from Section 4.2 and perform a small hyperparameter search over EMA smoothing, sampler choice, and L2 weight decay settings. During training, the worst-k domains are identified dynamically based on their domain-level training losses, with greater weight assigned to the k highest-loss domains, as defined in Equation 1. For evaluation, we compute the test F1 score for each domain and report the average F1 of the k lowest-performing domains.

Figure 4 reports the average F1 score of the k lowest-performing domains, denoted Min-k F1, together with macro and weighted F1. As k increases, Min-k F1 also increases because the metric is averaged over a progressively larger set that includes less challenging domains. However, the gains begin to plateau for approximately k = 5-7, suggesting diminishing benefits from emphasizing an increasingly large subset of domains. As k approaches $|\mathcal{D}|$, $\mathcal{L}_{\text{worst-}k}$ increasingly resembles uniform domain weighting and therefore loses its ability to target the weakest domains.

Macro and weighted F1 remain comparatively stable across values of k, with macro F1 ranging from $[0.827, 0.851]$ and weighted F1 ranging from $[0.773, 0.803]$. This stability indicates that practitioners can vary k to control how broadly robustness is distributed across weaker domains without substantially changing average-case performance. Overall, **smaller values of k provide more targeted optimization, whereas larger values distribute the robustness emphasis across a broader set of domains.**

7 CASE STUDY

We conclude with a practical case study on the Walmart–Amazon EM benchmark, following the price-based domain partitioning introduced in Section 1. We compare a baseline DITTO-trained DistilBERT model against RoDo-EM using the $\mathcal{L}_{\text{DRO}}$ objective to demonstrate the practical impact of robust optimization on an existing

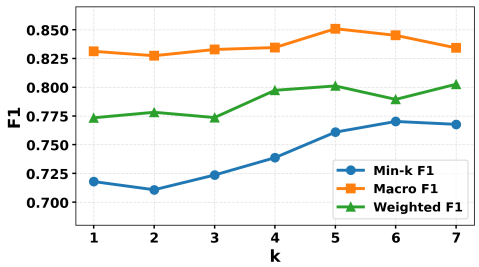


Figure 4: Minimum-k, macro, and weighted F1 scores across robust worst-k RoDo-EM models.

EM pipeline. As shown in Table 7, RoDo-EM improves F1 in the medium and high-price domains while maintaining competitive overall performance. Notably, all three domains achieve the target F1 threshold of 0.75, compared with two under the baseline model. These improvements also increase the *recovered value*, defined as the total representative price of correctly identified matching pairs, by \$498.81 (2.22%). Although simplified, this case study illustrates how improving worst-domain performance can yield practical benefits beyond aggregate matching metrics.

Table 7: Price-Aware Entity Matching Case Study

Model	High Price F1	Medium Price F1	Low Price F1	Weighted F1	Threshold Domains	Recovered Value
Base Model	0.7097	0.7970	**0.8198**	0.7892	66.7% (2/3)	\$22,474.43
RoDo-EM	**0.7879**	**0.8060**	0.7965	**0.7973**	**100% (3/3)**	**\$22,973.24**
Δ	+0.0782	+0.0090	−0.0233	+0.0081	+33.3%	+\$498.81 (+2.22%)

8 CONCLUSION

This paper introduced *Robust Entity Matching Across Domains* (REMAD), a new problem setting that extends traditional domain-aware Entity Matching by explicitly optimizing domain-level robustness. To address this problem, we presented **RoDo-EM**, a modular optimization framework that integrates robustness-aware objectives, domain-aware sampling strategies, and regularization techniques into the training pipeline of EM models. Through experiments on multiple domain-partitioned benchmarks, we demonstrated that RoDo-EM consistently improves lower-bound domain performance and domain parity while maintaining competitive overall matching performance, outperforming existing EM approaches across a variety of robustness objectives. Furthermore, our analyses showed that the framework integrates a suite of distinct objectives with configurable and interpretable tradeoffs between robustness, parity, and average-case performance. In future work, RoDo-EM could be extended to other backbone language models, including fine-tuned hosted LLMs and emerging foundation models for data management. More broadly, we hope this work motivates further research on robust optimization techniques for domain-aware data integration tasks beyond entity matching.

REFERENCES

[1] Mehdi Akbarian Rastaghi, Ehsan Kamalloo, and Davood Rafiei. 2022. Probing the robustness of pre-trained language models for entity matching. In *Proceedings of*

the 31st ACM International Conference on Information & Knowledge Management. 3786–3790.

[2] Tiago Brasileiro Araújo, Vasilis Efthymiou, and Kostas Stefanidis. 2025. Fairness and Explanations in Entity Resolution: An Overview. *IEEE Access* (2025).

[3] Jan-Philipp Awick and Jorge Marx Gómez. 2025. Entity Matching in the Era of Language Models: A Structured Literature Review. In *2025 25th International Conference on Control Systems and Computer Science (CSCS)*. IEEE, 368–375.

[4] Nils Barlaug and Jon Atle Gulla. 2021. Neural networks for entity matching: A survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15, 3 (2021), 1–37.

[5] Solon Barocas, Moritz Hardt, and Arvind Narayanan. 2023. *Fairness and machine learning: Limitations and opportunities*. MIT press.

[6] Aharon Ben-Tal, Dick Den Hertog, Anja De Waegenaere, Bertrand Melenberg, and Gijs Rennen. 2013. Robust solutions of optimization problems affected by uncertain probabilities. *Management Science* 59, 2 (2013), 341–357.

[7] Indrajit Bhattacharya and Lise Getoor. 2007. Collective entity resolution in relational data. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 1, 1 (2007), 5–es.

[8] Mikhail Bilenko and Raymond J Mooney. 2003. Adaptive duplicate detection using learnable string similarity measures. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 39–48.

[9] Alexander Brinkmann, Roee Shraga, and Christina Bizer. 2024. Sc-block: Supervised contrastive blocking within entity resolution pipelines. In *European Semantic Web Conference*. Springer, 121–142.

[10] Ursin Brunner and Kurt Stockinger. 2020. Entity matching with transformer architectures-a step forward in data integration. In *23rd International Conference on Extending Database Technology, Copenhagen, 30 March-2 April 2020*. OpenProceedings, 463–473.

[11] Xingguang Chen, Yimin Shi, and Xiaokui Xiao. 2026. Generalized Entity Matching with Adaptivity via Large Language Models. *Proc. ACM Manag. Data* 4, 3, Article 189 (May 2026), 25 pages. https://doi.org/10.1145/3802066

[12] Peter Christen. 2012. The data matching process. In *Data matching: concepts and techniques for record linkage, entity resolution, and duplicate detection*. Springer, 23–35.

[13] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. 2020. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys (CSUR)* 53, 6 (2020), 1–42.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[16] Cian Eastwood, Alexander Robey, Shashank Singh, Julius Von Kügelgen, Hamed Hassani, George J Pappas, and Bernhard Schölkopf. 2022. Probable domain generalization via quantile risk minimization. *Advances in Neural Information Processing Systems* 35 (2022), 17340–17358.

[17] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. 2018. Distributed representations of tuples for entity resolution. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1454–1467.

[18] Ivan P Fellegi and Alan B Sunter. 1969. A theory for record linkage. *Journal of the American statistical association* 64, 328 (1969), 1183–1210.

[19] Jiajie Fu, Haitong Tang, Arijit Khan, Sharad Mehrotra, Xiangyu Ke, and Yunjun Gao. 2025. In-context Clustering-based Entity Resolution with Large Language Models: A Design Space Exploration. *Proc. ACM Manag. Data* 3, 4, Article 252 (Sept. 2025), 28 pages. https://doi.org/10.1145/3749170

[20] Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The battleship approach to the low resource entity matching problem. *Proceedings of the ACM on Management of Data* 1, 4 (2023), 1–25.

[21] Bar Genossar, Roee Shraga, and Avigdor Gal. 2023. Flexer: Flexible entity resolution for multiple intents. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[22] Chaohui Guo, Michel CA Klein, and Zhisheng Huang. 2026. CaRL-EM: Cost-Aware Reinforcement Learning for Entity Matching with LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 27281–27293.

[23] Mauricio A Hernández and Salvatore J Stolfo. 1995. The merge/purge problem for large databases. *ACM Sigmod Record* 24, 2 (1995), 127–138.

[24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[25] Delaram Javdani, Hossein Rahmani, Milad Allahgholi, and Fatemeh Karimkhani. 2019. Deepblock: A novel blocking approach for entity resolution using deep learning. In *2019 5th International Conference on Web Research (ICWR)*. IEEE, 41–44.

[26] Jungo Kasai, Kun Qian, Sairam Gurajada, Yunyao Li, and Lucian Popa. 2019. Low-resource Deep Entity Resolution with Transfer and Active Learning. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 5851–5861.

[27] Hanna Köpcke, Andreas Thor, and Erhard Rahm. 2010. Evaluation of entity resolution approaches on real-world match problems. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 484–493.

[28] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment* 14, 1 (2020), 50–60.

[29] Evan Z Liu, Behzad Haghgoo, Annie S Chen, Aditi Raghunathan, Pang Wei Koh, Shiori Sagawa, Percy Liang, and Chelsea Finn. 2021. Just train twice: Improving group robustness without training group information. In *International Conference on Machine Learning*. PMLR, 6781–6792.

[30] Yuan Liu and Flavius Frasincar. 2025. FairNM: Fairness in Name Matching. In *International Conference on Applications of Natural Language to Information Systems*. Springer, 36–46.

[31] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[32] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. 2021. A survey on bias and fairness in machine learning. *ACM computing surveys (CSUR)* 54, 6 (2021), 1–35.

[33] Zhengjie Miao, Yuliang Li, and Xiaolan Wang. 2021. Rotom: A meta-learned data augmentation framework for entity matching, data cleaning, text classification, and beyond. In *Proceedings of the 2021 International Conference on Management of Data*. 1303–1316.

[34] Alvaro E Monge, Charles Elkan, et al. 1996. The field matching problem: algorithms and applications.. In *Kdd*, Vol. 2. 267–270.

[35] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 international conference on management of data*. 19–34.

[36] John Bosco Mugeni, Steven Lynden, Toshiyuki Amagasa, and Akiyoshi Matono. 2023. Adapterem: pre-trained language model adaptation for generalized entity matching using adapter-tuning. In *Proceedings of the 27th International Database Engineered Applications Symposium*. 140–147.

[37] John Bosco Mugeni, Steven Lynden, Toshiyuki Amagasa, and Akiyoshi Matono. 2024. MultiMatch: Low-Resource Generalized Entity Matching Using Task-Conditioned Hyperadapters in Multitask Learning. In *International Conference on Big Data Analytics and Knowledge Discovery*. Springer, 51–65.

[38] Ricardo Marçal de Andrade Nascimento, Daniel Xavier de Sousa, Guglielmo Faggioli, Paulo José Lage Alvarenga, Nicola Ferro, and Marcos André Gonçalves. 2025. A Robustness Assessment of Query Performance Prediction (QPP) Methods Based on Risk-Sensitive Analysis. In *Proceedings of the 2025 International ACM SIGIR Conference on Innovative Concepts and Theories in Information Retrieval (ICTIR)*. 416–429.

[39] Toan Nguyen, Kien Do, Bao Duong, and Thin Nguyen. 2024. Domain generalisation via risk distribution matching. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2790–2799.

[40] Kota Okayama, Hiroyoshi Ito, and Atsuyuki Morishima. 2026. Learning from unknown-unknowns: inconsistency-driven sampling for improving LLM entity matching. *Information Research an international electronic journal* 31, iConf (2026), 118–135.

[41] Ralph Peeters and Christian Bizer. 2021. Dual-objective fine-tuning of BERT for entity matching. *Proceedings of the VLDB Endowment* 14 (2021), 1913–1921.

[42] Ralph Peeters, Reng Chiz Der, and Christian Bizer. 2023. WDC Products: A Multi-Dimensional Entity Matching Benchmark. arXiv:2301.09521 [cs.LG] https://arxiv.org/abs/2301.09521

[43] Ralph Peeters, Aaron Steiner, and Christian Bizer. 2025. Entity matching using large language models. *OpenProceedings* 2 (2025), 529–541.

[44] Anna Primpeli, Ralph Peeters, and Christian Bizer. 2019. The WDC training dataset and gold standard for large-scale product matching. In *Companion Proceedings of The 2019 World Wide Web Conference*. 381–386.

[45] Nicholas Pulsone, Roee Shraga, and Gregory Goren. 2026. BEACON: Budget-Aware Entity Matching Across Domains (Extended Technical Report). *arXiv preprint* (2026). Technical report, available at arXiv.

[46] Nicholas Pulsone, Roee Shraga, and Gregory Goren. 2026. Understanding Domain-Aware Distribution Alignment in Budgeted Entity Matching. *arXiv preprint* (2026). Extended version, available at arXiv.

[47] Ricardo Trainotti Rabonato and Lilian Berton. 2025. A systematic review of fairness in machine learning. *AI and Ethics* 5, 3 (2025), 1943–1954.

[48] Shiori Sagawa, Pang Wei Koh, Tatsunori B Hashimoto, and Percy Liang. 2019. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *arXiv preprint arXiv:1911.08731* (2019).

[49] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[50] Sunita Sarawagi and Anuradha Bhamidipaty. 2002. Interactive deduplication using active learning. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 269–278.

[51] Nima Shahbazi, Nikola Danevski, Fatemeh Nargesian, Abolfazl Asudeh, and Divesh Srivastava. 2023. Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching. *Proc. VLDB Endow.* 16, 11 (July 2023), 3279–3292. https://doi.org/10.14778/3611479.3611525

[52] Aaron Steiner, Ralph Peeters, and Christian Bizer. 2025. Fine-tuning large language models for entity matching. In *2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW)*. IEEE, 9–17.

[53] Chenchen Sun, Yang Xu, Derong Shen, and Tiezheng Nie. 2024. Matching feature separation network for domain adaptation in entity matching. In *Proceedings of the ACM Web Conference 2024*. 1975–1985.

[54] Saravanan Thirumuruganathan, Shameem A Puthiya Parambath, Mourad Ouzzani, Nan Tang, and Shafiq Joty. 2018. Reuse and adaptation for entity resolution through transfer learning. *arXiv preprint arXiv:1809.11084* (2018).

[55] Mohamed Trabelsi, Jeff Heflin, and Jin Cao. 2022. Dame: Domain adaptation for matching entities. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1016–1024.

[56] Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Chengliang Chai, Guoliang Li, Ruixue Fan, and Xiaoyong Du. 2022. Domain adaptation for deep entity resolution. In *Proceedings of the 2022 international conference on management of data*. 443–457.

[57] Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Guoliang Li, Xiaoyong Du, Xiaofeng Jia, and Song Gao. 2023. Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration. *Proc. ACM Manag. Data* 1, 1 (2023), 84:1–84:26.

[58] Ellen M Voorhees. 2005. The TREC robust retrieval track. In *ACM SIGIR Forum*, Vol. 39. ACM New York, NY, USA, 11–20.

[59] Somin Wadhwa, Adit Krishnan, Runhui Wang, Byron C Wallace, and Luyang Kong. 2024. Learning from natural language explanations for generalizable entity matching. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 6114–6129.

[60] Jianghui Wang, Yang Chen, Xingyu Xie, Cong Fang, and Zhouchen Lin. 2023. Task-robust pre-training for worst-case downstream adaptation. *Advances in Neural Information Processing Systems* 36 (2023), 9458–9478.

[61] Jin Wang, Yuliang Li, Wataru Hirota, and Eser Kandogan. 2022. Machop: an end-to-end generalized entity matching framework. In *Proceedings of the Fifth International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. 1–10.

[62] Lidan Wang, Paul N Bennett, and Kevyn Collins-Thompson. 2012. Robust ranking models via risk-sensitive optimization. In *Proceedings of the 35th international ACM SIGIR conference on Research and development in information retrieval*. 761–770.

[63] Tianshu Wang, Xiaoyang Chen, Hongyu Lin, Xuanang Chen, Xianpei Han, Le Sun, Hao Wang, and Zhenyu Zeng. 2025. Match, Compare, or Select? An Investigation of Large Language Models for Entity Matching. In *COLING*. Association for Computational Linguistics, 96–109.

[64] Ziming Wang, Changwu Huang, Ke Tang, and Xin Yao. 2026. Procedural fairness in machine learning. *Journal of Artificial Intelligence Research* 85 (2026).

[65] William E Winkler. 1990. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage. (1990).

[66] Wen Xia, Hong Jiang, Dan Feng, Fred Douglis, Philip Shilane, Yu Hua, Min Fu, Yucheng Zhang, and Yukun Zhou. 2016. A comprehensive study of the past, present, and future of data deduplication. *Proc. IEEE* 104, 9 (2016), 1681–1710.

[67] Dezhong Yao, Yuhong Gu, Gao Cong, Hai Jin, and Xinqiao Lv. 2022. Entity Resolution with Hierarchical Graph Attention Networks. In *SIGMOD Conference*. ACM, 429–442.

[68] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. 2024. Jellyfish: Instruction-tuning local large language models for data preprocessing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8754–8782.

[69] Zeyu Zhang, Paul Groth, Iacer Calixto, and Sebastian Schelter. 2025. A Deep Dive Into Cross-Dataset Entity Matching with Large and Small Language Models. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*. OpenProceedings, 922–934. https://doi.org/10.48786/edbt.2025.75

Algorithm 1 RoDo-EM: Robust Domain-Aware Entity Matching

Require: Training data X partitioned into domains $\{X_1, \ldots, X_j\}$, validation data X_{val}, robustness objective $\mathcal{L}_{opt}$, backbone EM model f_θ
Ensure: Robust entity matching model f_{θ^*}
Initialize model parameters θ
Initialize domain statistics and best validation score
for each training epoch $t = 1, \ldots, T$ **do**
  Estimate or update domain-level losses $\{\mathcal{L}_1, \ldots, \mathcal{L}_j\}$
  Optionally update the robustness strength γ_{opt} from recent domain losses
  Construct the epoch training distribution {e.g., random, balanced, or domain-weighted sampling}
  for each minibatch B **do**
    Compute predictions $f_\theta(B)$
    Compute loss for each domain represented in B
    Combine domain losses using $\mathcal{L}_{opt}$ {e.g., Worst-k, DRO, Uniform, or Parity}
    Add regularization as configured
    Update θ by gradient descent
  end for
  Evaluate f_θ on validation domains
  if validation robustness objective improves **then**
    Save current parameters as θ^*
  end if
end for
Evaluate the selected model f_{θ^*} on $\mathcal{D}_{test}$
return f_{θ^*}

Table 8: Evaluation Metrics for REMAD.

Metric	Equation	Optimized by
Weighted F1	$\bar{F}_{\text{weighted}}(\theta) = \frac{\sum_{i\in\mathcal{D}} F_i(\theta)\,\lvert t_i\rvert}{\sum_{i\in\mathcal{D}} \lvert t_i\rvert}$	Baseline
Macro F1	$\bar{F}_{\text{macro}}(\theta) = \frac{1}{\lvert\mathcal{D}\rvert}\sum_{i\in\mathcal{D}} F_i(\theta)$	$\mathcal{L}_{\text{uniform}}$
Minimum-k F1	$\frac{1}{k}\sum_{i\in \text{Bottom}k_{i\in\mathcal{D}}(F_i(\theta))} F_i(\theta)$	$\mathcal{L}_{\text{worst-}k}$, $\mathcal{L}_{\text{DRO}}$
F1 Entropy	$-\frac{1}{\log\lvert\mathcal{D}\rvert}\sum_{i\in\mathcal{D}} q_i(\theta)\log q_i(\theta)$	$\mathcal{L}_{\text{par}}(\theta; m_{\text{entropy}})$
F1 Variance	$\frac{1}{\lvert\mathcal{D}\rvert}\sum_{i\in\mathcal{D}}\left(F_i(\theta)-\bar{F}(\theta)\right)^2$	$\mathcal{L}_{\text{par}}(\theta; m_{\text{var}})$
PPVP Variance	$\frac{1}{\lvert\mathcal{D}\rvert}\sum_{i\in\mathcal{D}}\left(P_i(\theta)-\bar{P}(\theta)\right)^2$	$\mathcal{L}_{\text{par}}(\theta; m_{\text{PPVP}})$
TPRP Variance	$\frac{1}{\lvert\mathcal{D}\rvert}\sum_{i\in\mathcal{D}}\left(R_i(\theta)-\bar{R}(\theta)\right)^2$	$\mathcal{L}_{\text{par}}(\theta; m_{\text{TPRP}})$

A RoDo-EM Algorithm

Algorithm 1 provides a high-level overview of the RoDo-EM training procedure described in Section 4.4. It summarizes the integration of robustness optimization, domain-aware sampling, regularization, and validation-based model selection into a unified EM framework. Here we additionally define the formulas for computing each of the evaluation metrics for RoDo-EM in Table 8

Table 10: Validation-selected adaptive sliding-window configurations across datasets and robustness objectives. Window size (w), update granularity, and update frequency are selected independently for each dataset and objective.

Dataset	Optimizer	w	Granularity	Frequency	Objective Score
WDC-MD	DRO	5	Iteration	1	0.667
	Entropy	3	Epoch	1	0.998
	PPVP	1	Epoch	1	0.009
WDC4	DRO	10	Iteration	5	0.814
	Entropy	3	Epoch	2	1.000
	PPVP	5	Iteration	1	0.0004
Abt-Buy	DRO	1	Epoch	1	0.750
	Entropy	1	Epoch	1	0.999
	PPVP	10	Iteration	1	0.003

B Adaptive Robust Weighting Ablation

Table 9: Comparison of robustness weighting strategies across three domain-partitioned EM benchmarks.

(a) WDC-MD (50% cc, 50% unseen)

Model	Min. ↑	Macro ↑	Weighted ↑	F1 Entropy ↑	PPVP Var. ↓
Base (GEN Ditto)	0.421	0.815	**0.810**	0.993	0.026
RoDo-EM (Base Adp.)	**0.714**	**0.854**	**0.810**	0.997	**0.008**
RoDo-EM (Adp. SW)	0.706	**0.854**	**0.810**	**0.998**	0.009
RoDo-EM (Static Wt.)	**0.714**	**0.854**	**0.810**	0.997	**0.008**

(b) WDC Product Corpus

Model	Min. ↑	Macro ↑	Weighted ↑	F1 Entropy ↑	PPVP Var. ↓
Base (GEN Ditto)	0.814	0.864	**0.874**	0.999	0.001
RoDo-EM (Base Adp.)	0.857	**0.869**	**0.874**	**1.000**	**0.000**
RoDo-EM (Adp. SW)	0.814	**0.869**	**0.874**	**1.000**	**0.000**
RoDo-EM (Static Wt.)	**0.859**	**0.869**	**0.874**	**1.000**	**0.000**

(c) Abt-Buy

Model	Min. ↑	Macro ↑	Weighted ↑	F1 Entropy ↑	PPVP Var. ↓
Base (GEN Ditto)	0.333	0.796	**0.810**	0.993	0.043
RoDo-EM (Base Adp.)	**0.750**	**0.884**	**0.810**	0.997	**0.003**
RoDo-EM (Adp. SW)	**0.750**	**0.884**	**0.810**	**0.999**	**0.003**
RoDo-EM (Static Wt.)	0.667	**0.884**	**0.810**	0.996	**0.003**

In our main experiments, RoDo-EM employs an adaptive weighting strategy that updates the robustness weight for each objective (e.g., γ_{DRO} or γ_{par}) at the beginning of each training epoch using domain-level losses from the preceding epoch. This approach adjusts the robustness strength according to the relative difficulty of the worst-performing domain. For example, when domain losses are relatively uniform, a smaller robustness weight may be sufficient, whereas a larger weight may be beneficial when one domain exhibits substantially higher loss than the others.

In this section, we investigate the alternative adaptive weighting configurations introduced in Section 4.2. Specifically, we vary three parameters: (1) update granularity, which determines whether domain-level losses are measured at the epoch or iteration level; (2) window size, which specifies the number of preceding observations used to compute the adaptive weight; and (3) update frequency, which controls how often the weight is recomputed. We perform

a grid search over update granularities {epoch, iteration}, window sizes $w \in \{1, 3, 5, 10\}$, and update frequencies $\{1, 2, 5, 10\}$.

Table 10 summarizes the configurations selected using validation performance for each dataset and robustness objective. The selected configurations vary across datasets and objectives, indicating that no single sliding-window configuration consistently yields the best performance. However, an update frequency of 1 is selected in most cases, suggesting that relatively frequent adjustments to the robustness weight may be beneficial as domain-level losses evolve during training.

We next compare three robustness weighting strategies against the standard non-robust GEN baseline: (1) RoDo-EM with the base adaptive weighting strategy used throughout our main experiments (Base Adp.); (2) RoDo-EM with the validation-selected sliding-window configuration from Table 10 (Adp. SW); and (3) RoDo-EM with a fixed robustness weight selected through a grid search over $\{2.0, 4.0\}$ (Static Wt.). Table 9 reports the resulting performance across the three benchmarks.

Overall, no single weighting strategy consistently outperforms the others. On WDC-MD, the base adaptive and static weighting strategies achieve identical performance across all reported metrics. A similar pattern emerges on the WDC Product Corpus, where static weighting achieves a slightly higher minimum domain F1 score than the base adaptive strategy, while the remaining metrics are comparable. On Abt-Buy, the sliding-window strategy achieves the highest F1 entropy while matching the base adaptive strategy on the remaining metrics. **These results suggest that the alternative weighting strategies offer comparable robustness improvements, with their relative effectiveness varying across datasets and objectives.**

Given the absence of a consistently superior alternative, we retain the base adaptive weighting strategy for our main experiments. This configuration achieves competitive robustness performance without requiring additional hyperparameter tuning for the robustness weight or sliding-window parameters. In contrast, static weighting requires selecting an objective-specific robustness weight, while sliding-window weighting introduces additional choices for window size, update granularity, and update frequency. Consequently, the base adaptive strategy offers a favorable balance between robustness performance and configuration complexity, making it our recommended default for RoDo-EM.

C Comparing Batch Sampling Techniques

In this section we investigate the effect of the domain-aware batch sampler. Table 11 compares three sampling strategies for $\mathcal{L}_{\text{worst-1}}$: the default random sampler, a balanced sampler that enforces approximately equal domain representation within each batch, and a loss-based sampler that oversamples domains with higher losses from previous training iterations. All remaining hyperparameters are fixed to the values used in the main experiments.

The loss-based sampler achieves the highest minimum domain F1 and macro F1 scores, while also improving weighted F1 relative to the standard random sampler. The balanced sampler similarly improves macro and weighted F1 while maintaining the same minimum domain F1 as the random sampler. Notably, balanced sampling achieves the highest weighted F1 among the evaluated strategies, suggesting a favorable tradeoff between domain-level robustness and overall matching performance. Accordingly, we retain balanced sampling as the default configuration in our main experiments, while noting that loss-based sampling may be preferable when prioritizing worst-domain performance.

Table 11: Impact of sampling methods on $\mathcal{L}_{\text{worst-1}}$ optimization on WDC-MD.

Sampling Method	Min. F1	Macro F1	Weighted F1
None	0.6000	0.7881	0.7251
Balanced	0.6000	0.8139	**0.7838**
Loss-based	**0.7143**	**0.8224**	0.7721